

The Anti-Slop Development Stack



A battle-tested development methodology for building frontend projects with zero tolerance for low-quality code. This stack enforces quality at every layer — from how you define features to how you commit code.

What is slop? Code that looks productive but adds no value: premature abstractions, dead code, over-engineering, “just in case” fallbacks, untested functions, and patterns applied without reason.

What this stack does: Makes it structurally impossible for slop to survive. Every layer catches a different class of problems, and nothing ships without passing all of them.

Table of Contents

1. [Stack Overview](#)
 2. [Layer 0: Project Scaffold](#)
 3. [Layer 1: Agent Configuration](#)
 4. [Layer 2: Linting & Formatting](#)
 5. [Layer 3: Testing Infrastructure](#)
 6. [Layer 4: Spec-Driven Development](#)
 7. [Layer 5: TDD Implementation](#)
 8. [Layer 6: Mutation Testing](#)
 9. [Layer 7: Component Testing](#)
 10. [Layer 8: Git Hooks – The Gate](#)
 11. [Layer 9: CI Pipeline](#)
 12. [The Workflow in Practice](#)
 13. [Agent Rules Reference](#)
 14. [Config Files Reference](#)
-

1. Stack Overview

Layer 0: Project Scaffold	- Framework + TypeScript + Bun
Layer 1: Agent Configuration	- AGENTS.md, rules, symlinks
Layer 2: Linting & Formatting	- Oxlint (anti-slop rules) + Oxfmt + reac
Layer 3: Testing Infrastructure	- Vitest + fast-check + Testing Library
Layer 4: Spec-Driven Development	- OpenSpec (proposal → design → specs → t
Layer 5: TDD Implementation	- Red/Green/Refactor cycle
Layer 6: Mutation Testing	- Stryker (proves your tests actually tes
Layer 7: Component Testing	- Playwright CT (real browser, real rende
Layer 8: Git Hooks - The Gate	- Lefthook (all checks must pass before c
Layer 9: CI Pipeline	- GitHub Actions (the final enforcer)

Key principle: Set up ALL architecture layers FIRST, before writing any feature code. Then use OpenSpec to define features, then implement with the full stack enforcing quality at every step.

2. Layer 0: Project Scaffold

Start with your framework's official scaffold. Don't hand-roll a project setup.

Example: React + TanStack Start

```
cd frontend/
bunx create-start my-project
cd my-project/
```

Post-scaffold cleanup

1. **Switch to Bun** (if scaffold used npm/pnpm): Delete `package-lock.json` or `pnpm-lock.yaml`, run `bun install`
2. **Remove demo files** — delete scaffold boilerplate components, demo routes, example pages
3. **Remove scaffold tooling you'll replace** — delete `eslint.config.js`, `prettier.config.js`, `.prettierrignore` (replaced by Oxlint + Oxfmt in Layer 2)
4. **Configure TypeScript strict mode** — ensure `tsconfig.json` has `strict: true`, `noUnusedLocals: true`, `noUnusedParameters: true`
5. **Configure Vite** — set base path, add plugins (TailwindCSS, React, etc.)

6. **Simplify remaining files** — strip demo content from routes, leave a working but minimal app

Verify

```
bun dev    # App loads
bun run build # Build succeeds
```

3. Layer 1: Agent Configuration

AI agents (Claude Code, Codex, etc.) need rules to follow. Without explicit rules, they generate slop.

Folder structure

```
.agent/
  rules/
    tdd.md           - Red/green/refactor protocol
    anti-slop.md     - Code quality rules
    testing.md       - Testing standards and tools
    code-style.md    - TypeScript strict, complexity limits
    commits.md       - Commit workflow
  skills/            - Agent skills (OpenSpec, react-doctor, etc.)
  workflows/         - OpenSpec workflows
  commands/          - OpenSpec commands
  AGENTS.md          - Real file (source of truth, project overview)
  CLAUDE.md → AGENTS.md - Symlink (Claude Code reads this)
  .claude → .agent    - Symlink (Claude Code reads this)
  .codex → .agent     - Symlink (OpenAI Codex reads this)
```

Why symlinks?

Different AI agents look for different files:

- **Claude Code** reads CLAUDE.md and .claude/
- **OpenAI Codex** reads AGENTS.md and .codex/

- **Others** — add symlinks as needed

One source of truth (`.agent/`), multiple entry points.

Create the structure

```
mkdir -p .agent/rules .agent/skills

# Create AGENTS.md with project overview, tech stack, commands
# (see Agent Rules Reference section for content)

# Symlinks
ln -s AGENTS.md CLAUDE.md
ln -s .agent .claude
ln -s .agent .codex
```

What goes in [AGENTS.md](#)

- Project overview (what this project does)
- Repository structure (where things live)
- Technology stack (every tool and version)
- Available commands (every script in package.json)
- Reference to `.agent/rules/` files
- CI pipeline summary
- Deferred/not-yet-configured items

4. Layer 2: Linting & Formatting

Oxlint is 50-100x faster than ESLint. Oxfmt replaces Prettier. Both are Rust-based.
react-doctor catches React-specific anti-patterns that linters miss.

Install

```
bun add -d oxlint oxfmt lefthook
```

react-doctor

react-doctor audits React code for best-practice violations that static linters cannot catch — hook rule violations, component anti-patterns, performance issues, and accessibility gaps. It runs on every commit via lefthook and produces a scored report.

```
bunx react-doctor@latest .          # Full project audit
bunx react-doctor@latest . --diff   # Only changed files (use in hooks)
```

No config file needed — it works out of the box. Runs via `bunx` so it's always the latest version.

.oxlintrc.json — Anti-slop rules

These rules are the core of slop prevention. Every rule has a reason:

```
{
  "$schema": "node_modules/oxlint/configuration_schema.json",
  "plugins": ["react", "react-perf", "typescript", "jsx-a11y"],
  "rules": {
    "react/react-in-jsx-scope": "off",
    "typescript/no-explicit-any": "warn",
    "eslint/complexity": ["error", {"max": 10}],
    "eslint/max-lines-per-function": ["error", {"max": 50, "skipComments": true}],
    "eslint/max-lines": ["error", {"max": 250, "skipBlankLines": true, "skipComments": true}],
    "eslint/max-params": ["error", 3],
    "eslint/max-depth": ["error", 3],
    "eslint/max-statements": ["error", 25],
    "eslint/max-classes-per-file": ["error", 1]
  },
  "ignorePatterns": ["*.html", "docker", "public", "__tests__", "*.test.*"]
}
```

Why each rule matters:

Rule	Limit	Why
complexity	10	Forces simple, readable logic paths
max-lines-per-function	50	If a function is longer, it does too much — split it

Rule	Limit	Why
max-params	4	More than 4 params = use an options object
max-depth	4	Deep nesting = extract into functions
max-statements	15	Too many statements = function does too much
no-unused-vars	error	Dead code is slop
no-explicit-any	error	any disables TypeScript — use unknown or generics

Test files are exempted from max-lines-per-function and max-statements because test describe blocks naturally accumulate many statements.

.oxfmttrc.json

```
{
  "$schema": "./node_modules/oxfmt/configuration_schema.json",
  "printWidth": 140,
  "tabWidth": 2,
  "useTabs": false,
  "semi": true,
  "singleQuote": false,
  "trailingComma": "es5",
  "arrowParens": "always",
  "bracketSpacing": false,
  "endOfLine": "lf"
}
```

Package.json scripts

```
{
  "lint": "oxlint --max-warnings=0",
  "lint:fix": "oxlint --fix",
  "format": "oxfmt",
  "prepare": "lefthook install"
}
```

`--max-warnings=0` is critical. It means ANY warning is a build failure. No “we’ll fix it later.”

Verify

```
bun lint      # 0 warnings, 0 errors
bun format   # Formats all files
```

5. Layer 3: Testing Infrastructure

Four testing tools, each catching different problems:

Tool	What it catches	When to use
Vitest	Broken logic, regressions	Every function, every component
fast-check	Edge cases humans miss	Pure functions with well-defined inputs
react-doctor	React anti-patterns, hook violations	Every commit with React code (Layer 2)
Stryker	Tests that don’t actually test anything	After tests pass (Layer 6)
Playwright CT	Rendering bugs in real browsers	UI components (Layer 7)

Install

```
# Vitest + Testing Library + fast-check
bun add -d vitest @vitest/coverage-v8 @vitest/ui @testing-library/react @te

# Stryker (mutation testing)
npx stryker init
# Select: vitest, typescript checker
```

```
# Playwright CT (component testing)
bun add -d @playwright/experimental-ct-react playwright
npx playwright install chromium
```

vitest.config.ts

```
import {defineConfig} from "vitest/config";
import {resolve} from "path";

export default defineConfig({
  test: {
    globals: true,
    environment: "jsdom",
    pool: "forks",
    include: ["src/**/*.{test,spec}.{js,ts,jsx,tsx}"],
    setupFiles: ["./vitest.setup.ts"],
    css: true,
    coverage: {
      provider: "v8",
      reporter: ["text", "json", "json-summary", "html", "lcov"],
      cleanOnRerun: true,
      exclude: [
        "node_modules/", "dist/", "**/*.config.{js,ts}",
        "**/types.ts", "**/*.d.ts", "**/index.ts",
        "**/__tests__/**", "**/*.gen.ts",
      ],
      include: ["src/**/*.{js,ts,jsx,tsx}"],
      thresholds: {
        lines: 90,
        functions: 90,
        branches: 90,
        statements: 90,
      },
    },
  },
  resolve: {
    alias: {"@": resolve(__dirname, "./src")},
  },
});
```

90% thresholds from day one. Don't start at 80% and "raise it later" — you won't.

vitest.setup.ts

```
import "@testing-library/jest-dom/vitest";
import {cleanup} from "@testing-library/react";
import {afterEach} from "vitest";

afterEach(() => {
  cleanup();
});

// Mock localStorage for jsdom
const localStorageMock = (() => {
  let store: Record<string, string> = {};
  return {
    getItem: (key: string) => store[key] ?? null,
    setItem: (key: string, value: string) => { store[key] = value; },
    removeItem: (key: string) => { delete store[key]; },
    clear: () => { store = {}; },
  };
})();
Object.defineProperty(window, "localStorage", {value: localStorageMock});
```

stryker.config.json

```
{
  "$schema": "./node_modules/@stryker-mutator/core/schema/stryker-schema.js",
  "packageManager": "npm",
  "reporters": ["html", "clear-text", "progress"],
  "testRunner": "vitest",
  "coverageAnalysis": "perTest",
  "mutate": [
    "src/lib/**/*.ts",
    "src/components/**/*.ts",
    "src/components/**/*.tsx",
    "!src/**/*.test.*",
    "!src/**/*.spec.*",
    "!src/**/*.ct.*",
    "!src/**/*.gen.ts",
    "!src/**/*.d.ts"
  ],
  "checkers": ["typescript"],
  "tsconfigFile": "tsconfig.json",
  "thresholds": {
```

```

    "high": 90,
    "low": 80,
    "break": 80
  },
  "tempDirName": ".stryker-tmp",
  "ignorePatterns": [".codex", ".claude", ".agent", "openspec", "AGENTS.md"]
}

```

Important: The `ignorePatterns` must include symlinks (`.codex` , `.claude`) and agent folders. Stryker tries to copy them to its sandbox and fails on symlinks.

Important: Scope mutate to `src/lib/` and `src/components/` — not `src/**/*.Route` files and config files don't need mutation testing.

playwright-ct.config.ts

```

import {defineConfig, devices} from "@playwright/experimental-ct-react";

export default defineConfig({
  testDir: "./src",
  testMatch: "**/*.ct.tsx",
  timeout: 10_000,
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: "html",
  use: {
    trace: "on-first-retry",
    ctPort: 3100,
  },
  projects: [
    {name: "chromium", use: {...devices["Desktop Chrome"]}},
  ],
});

```

All scripts

```

{
  "test": "vitest run",
  "test:ui": "vitest --ui",
  "test:coverage": "vitest --coverage",

```

```

    "stryker": "stryker run",
    "stryker:incremental": "stryker run --incremental",
    "test:ct": "playwright test -c playwright-ct.config.ts",
    "test:ct:ui": "playwright test -c playwright-ct.config.ts --ui"
  }

```

stryker:incremental stores results and reuses them on subsequent runs. Use this for per-commit checks. Use **stryker (full)** for CI and periodic full scans.

File locations

```

src/
  lib/
    __tests__/
      video-utils.test.ts      ← Vitest + fast-check
  components/
    __tests__/
      VideoError.test.tsx      ← Vitest + Testing Library
      VideoError.ct.tsx        ← Playwright CT
  error/
    __tests__/
      ErrorLayout.test.tsx     ← Vitest + Testing Library
      ErrorLayout.ct.tsx       ← Playwright CT

```

6. Layer 4: Spec-Driven Development

Don't start coding. Start with specs. OpenSpec creates structured change proposals that become your implementation roadmap.

Install OpenSpec

```
npx openspec init
```

This creates `.agent/skills/`, `.agent/workflows/`, and `.agent/commands/` for OpenSpec.

The OpenSpec workflow

Every feature follows this sequence:

1. `/openspec-propose` → Creates: `proposal.md`, `design.md`, `specs/*.md`, `tasks`
2. `/openspec-apply` → Implements tasks one by one (TDD)
3. `/openspec-archive` → Archives completed change

What each artifact contains

Artifact	Purpose	Content
<code>proposal.md</code>	Why this change exists	Motivation, what changes, capabilities, impact
<code>design.md</code>	How to implement it	Decisions with rationale, alternatives considered, risks
<code>specs/*.md</code>	What the system should do	EARS requirements with WHEN/THEN scenarios
<code>tasks.md</code>	Implementation checklist	Ordered tasks with checkboxes, grouped logically

Split features into small changes

Bad: One change with 24 tasks covering the entire project.

Good: 5-6 focused changes with 4-8 tasks each.

Each change gets its own proposal, design, specs, and tasks. This gives you:

- Clear boundaries between features
- Focused code reviews
- Independent verification
- Progress visibility

Example: Video embed project

```
video-utils           (4 tasks) - Pure utility functions
error-components      (4 tasks) - ErrorLayout + VideoError
navigation-components (4 tasks) - NotFound + DefaultCatchBoundary
```

routing-setup	(7 tasks) – Routes, layouts, assets
video-player	(3 tasks) – Main VideoIframe component
quality-gates	(8 tasks) – CT tests, coverage, mutation testing

Spec format (EARS)

ADDED Requirements

Requirement: Extract status from error.status property

The system SHALL return the numeric value of `error.status` when the error object has a numeric `status` property.

Scenario: Error with numeric status

- **WHEN** an error object has a numeric `status` property (e.g., `{ status`
- **THEN** `getHttpStatus` SHALL return that number

Every scenario is a test case. If you can't write a test for it, the spec is too vague.

7. Layer 5: TDD Implementation

Every task from OpenSpec follows the red/green/refactor cycle.

The cycle

1. RED – Write a failing test that describes the desired behavior
2. GREEN – Write the minimum code to make the test pass
3. REFACTOR – Clean up without changing behavior
4. VERIFY – Run ALL checks (lint, test, stryker, CT if applicable)

Who writes the test?

Either AI or human. What matters is:

- The test exists BEFORE the implementation
- The test fails for the right reason (import error or assertion failure, not syntax error)

- The test clearly expresses the expected behavior

Example: Pure function (getHttpStatus)

RED — Write failing tests:

```
import {describe, it, expect} from "vitest";
import fc from "fast-check";
import {getHttpStatus} from "../video-utils";

describe("getHttpStatus", () => {
  // Example-based tests
  it("returns status when error has numeric status property", () => {
    expect(getHttpStatus({status: 404})).toBe(404);
  });

  it("returns null for null input", () => {
    expect(getHttpStatus(null)).toBeNull();
  });

  // Property-based tests (fast-check)
  it("returns status for any object with numeric status in 400-599", () => {
    fc.assert(
      fc.property(fc.integer({min: 400, max: 599}), (status) => {
        expect(getHttpStatus({status})).toBe(status);
      }),
    );
  });

  it("returns null for any primitive value", () => {
    fc.assert(
      fc.property(
        fc.oneof(fc.string(), fc.integer(), fc.boolean()),
        (value) => {
          expect(getHttpStatus(value)).toBeNull();
        },
      ),
    );
  });
});
```

Run tests — they fail (file doesn't exist yet). Good.

GREEN – Implement just enough:

```
interface HttpError extends Error {
  status?: number;
  statusCode?: number;
}

export function getHttpStatus(error: unknown): number | null {
  if (error && typeof error === "object") {
    const err = error as HttpError;
    if (typeof err.status === "number") return err.status;
    if (typeof err.statusCode === "number") return err.statusCode;
    if (err.message) {
      const match = err.message.match(/\b(4\d{2}|5\d{2})\b/);
      if (match) return parseInt(match[1], 10);
    }
  }
  return null;
}
```

Run tests – they pass. Good.**VERIFY – Run everything:**

```
bun lint # 0 warnings
bun run test # All pass
bun run stryker:incremental # 80%+ mutation score
bun run test:ct # All pass (if UI component)
bunx react-doctor . --diff # 100/100 (if React code)
```

When to use which test type

Code type	Vitest	fast-check	react-doctor	Playwright CT
Pure functions	Yes	Yes	No	No
React components	Yes	No	Yes	Yes (presentational)
Hooks	Yes	Maybe	Yes	No
Routes/config	Yes	No	No	No

Code type	Vitest	fast-check	react-doctor	Playwright CT
Components with SDK deps	Yes (mock)	No	Yes	No (hard to mount)

fast-check is for pure functions with well-defined input domains. Don't force it on components.

Playwright CT is for presentational components that render without complex context. Don't use it for components that need SDK mocking or router context.

8. Layer 6: Mutation Testing

Tests can pass and still be worthless. Stryker proves your tests actually catch bugs.

What Stryker does

1. Takes your production code
2. Makes small changes (mutations): `===` → `!==`, `true` → `false`, delete a line
3. Runs your tests against each mutation
4. If tests still pass with the mutation → your tests are weak (mutant **survived**)
5. If tests fail → your tests caught it (mutant **killed**)

Interpreting results

```
File          | % Mutation score | # killed | # timeout | # survived | #
video-utils.ts | 100.00          | 1       | 14       | 0         |
```

- **Killed:** Tests caught the mutation. Good.
- **Timeout:** Mutation caused infinite loop. Effectively killed. Good.
- **Survived:** Tests didn't catch the mutation. BAD — strengthen your tests.
- **No coverage:** Code isn't tested at all. BAD — write tests.
- **Errors:** Mutation caused compile error (TypeScript caught it). Good — TypeScript is a layer of defense.

Thresholds

- **80% mutation score** = minimum. Below this, the build breaks.
- **90%+** = target. Aim for this.
- **100%** = achievable on small, well-tested modules.

Fix surviving mutants

Don't suppress them with ignore comments. Strengthen the tests:

```
// If Stryker shows this mutant survived:
//   - if (status === 404) {
//   + if (status !== 404) {

// Your test is probably:
it("handles 404", () => {
  // ...only checks the 404 path, never checks non-404
});

// Fix: Add a test for the non-404 path too
it("handles non-404 errors", () => {
  // ...explicitly tests the other branch
});
```

Run commands

```
bun run stryker          # Full run (CI, periodic)
bun run stryker:incremental # Incremental (per-commit, faster)
```

9. Layer 7: Component Testing

Playwright CT mounts React components in a real Chromium browser. This catches rendering bugs that jsdom misses.

When to use Playwright CT

- **Use for:** Presentational components (ErrorLayout, VideoError, NotFound)
- **Don't use for:** Components that need SDK mocking or complex router context
- **Don't use for:** Pure functions (use Vitest + fast-check)

Example CT test

```
// src/components/__tests__/VideoError.ct.tsx
import {test, expect} from "@playwright/experimental-ct-react";
import {VideoError} from "../VideoError";

test("not-found variant shows Video Not Found", async ({mount}) => {
  const component = await mount(<VideoError type="not-found" />);
  await expect(component.getByText("Video Not Found")).toBeVisible();
});

test("load-error variant shows Unable to Load Video", async ({mount}) => {
  const component = await mount(<VideoError type="load-error" />);
  await expect(component.getByText("Unable to Load Video")).toBeVisible();
});
```

Run

```
bun run test:ct      # Run all CT tests
bun run test:ct:ui   # Interactive UI mode
```

10. Layer 8: Git Hooks – The Gate

Nothing gets committed without passing every check. Lefthook runs them all in parallel.

lefthook.yml

```
pre-commit:
  parallel: true
  commands:
    lint:
      glob: "*. {js,ts,jsx,tsx}"
```

```
run: bun lint {staged_files}
stage_fixed: true
format:
  glob: "*. {js,ts,jsx,tsx,json,md}"
  run: bun format {staged_files}
  stage_fixed: true
test:
  glob: "*. {js,ts,jsx,tsx}"
  run: bun run test --related {staged_files} --run
react-doctor:
  glob: "*. {jsx,tsx}"
  run: bunx -y react-doctor@latest . --diff
test-ct:
  glob: "*. {jsx,tsx}"
  run: bun run test:ct
stryker:
  glob: "*. {ts,tsx}"
  run: bun run stryker:incremental
```

What each hook does

Hook	What it checks	Fails if
lint	Oxlint anti-slop rules	Any warning (-max-warnings=0)
format	Code formatting	Files aren't formatted (auto-fixes)
test	Vitest (related files only)	Any test fails
react-doctor	React best practices	Any issue found
test-ct	Playwright CT	Any component test fails
stryker	Mutation testing (incremental)	Score below 80%

All run in parallel — faster than sequential. If any one fails, the commit is rejected.

Before every commit (manual verification)

Even with hooks, verify manually during development:

```
bun lint # 0 warnings
bunx -y react-doctor@latest . --diff # 100/100
bun run test # All pass
bun run stryker:incremental # 80%+ mutation score
bun run test:ct # All CT tests pass (if applicable)
```

11. Layer 9: CI Pipeline

The final enforcer. Even if someone bypasses git hooks, CI catches it.

.github/workflows/publish.yml

```
name: Build and Release

on:
  workflow_dispatch:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Bun
        uses: oven-sh/setup-bun@v2

      - name: Install dependencies
        run: bun install --frozen-lockfile
```

- **name:** Lint (zero warnings gate)
run: bun lint
- **name:** React best practices audit
run: bunx react-doctor@latest . --diff
- **name:** Build
run: bun run build
- **name:** Run tests with coverage
run: bun run test:coverage --run
- **name:** Report coverage to PR
if: github.event_name == 'pull_request'
uses: davelosert/vitest-coverage-report-action@v2
with:
 - json-summary-path:** ./coverage/coverage-summary.json
 - json-final-path:** ./coverage/coverage-final.json
 - vite-config-path:** ./vitest.config.ts
- **name:** Mutation testing (changed files)
if: github.event_name == 'pull_request'
run: bun run stryker:incremental
- **name:** Release Please
if: github.event_name == 'push' && github.ref == 'refs/heads/main'
uses: googleapis/release-please-action@v4

Quality gates

Gate	Threshold	Blocks
Lint	0 warnings	PR merge
react-doctor	No issues on changed files	PR merge
Build	Must succeed	PR merge
Test coverage	90%+ (lines, functions, branches, statements)	PR merge
Mutation score	80%+ on changed files	PR merge

Playwright CT stays local-only — adding browser install to CI is slow and not worth it for most projects.

12. The Workflow in Practice

Here's the complete flow for adding a feature to a project with this stack:

Phase 1: Define (before any code)

1. `/openspec-propose <feature-name>`
 - Creates `proposal.md`, `design.md`, `specs/*.md`, `tasks.md`
2. Review the artifacts
 - Are the specs testable? Are the tasks small enough?
 - Adjust if needed
3. Commit the specs
 - `chore: add openspec for <feature-name>`

Phase 2: Implement (TDD for each task)

4. `/openspec-apply <feature-name>`
 - For each task:
 - a. RED: Write failing test
 - Example-based tests (Vitest)
 - Property-based tests (fast-check) for pure functions
 - Run: `bun run test` → FAILS ✓
 - b. GREEN: Implement minimum code
 - Just enough to pass the test
 - Run: `bun run test` → PASSES ✓
 - c. VERIFY: Run full stack
 - `bun lint` → 0 warnings ✓
 - `bun run test` → all pass ✓
 - `bun run stryker:incremental` → 80%+ ✓
 - `bun run test:ct` → all pass ✓ (if UI component)
 - `bunx react-doctor . --diff` → 100/100 ✓ (if React code)
 - d. COMMIT

- feat: add <what was implemented>
- Lefthook runs all hooks automatically

Phase 3: Quality verification

5. Run full coverage check
→ `bun run test:coverage` → 90%+ on all metrics
6. Run full Stryker
→ `bun run stryker` → 80%+ mutation score
7. Write Playwright CT tests (for UI components)
→ `bun run test:ct` → all pass
8. `/openspec-archive <feature-name>`
→ Archives the completed change

Commit discipline

- **One logical change per commit**
- **Conventional commits:** feat:, fix:, refactor:, test:, docs:, chore:
- **Every commit passes all hooks** — lint, format, test, react-doctor, CT, stryker
- **Step by step:** finish one thing → verify → commit → move on

13. Agent Rules Reference

These files go in `.agent/rules/`. Each has `trigger: always_on` so AI agents always follow them.

[tdd.md](#)

TDD Workflow

Core Protocol

Every new feature or bug fix follows the red/green/refactor cycle:

1. ****Red**** – AI or human writes a failing test that describes the desired behavior
2. ****Green**** – Write the minimum code to make the test pass
3. ****Refactor**** – Clean up without changing behavior, tests stay green

Rules

- No production code without a corresponding test
- Tests describe behavior, not implementation
- One assertion per test when possible
- Test names read as specifications: "returns null for non-object input"
- Property-based tests (fast-check) for pure functions with well-defined domain
- Mock external dependencies, not internal logic

Who Writes the Test

Either AI or human can write the failing test first. What matters is:

- The test exists before the implementation
- The test fails for the right reason (not a syntax error)
- The test clearly expresses the expected behavior

[anti-slop.md](#)

Anti-Slop Rules

What Is Slop

Slop is code that looks productive but adds no value: premature abstraction, dead code, over-engineering, "just in case" fallbacks, and patterns applied without reason.

Rules

- ****No dead code**** – delete unused functions, variables, imports, and types
- ****No premature abstraction**** – extract only when there is real duplication
- ****No "just in case" code**** – don't add error handling for scenarios that don't happen
- ****No TODO/FIXME without a linked issue**** – if it matters, track it. If not, delete it
- ****No magic numbers or strings**** – use named constants for values that have meaning
- ****Functions do one thing**** – if a function name needs "and" to describe it, refactor
- ****Naming must be descriptive**** – a reader should understand what a function does
- ****No feature flags or backwards-compatibility shims**** – just change the code
- ****No unnecessary comments**** – only comment why, never what

[testing.md](#)

Testing Standards

Tools

Tool	Purpose	Command
-----	-----	-----
Vitest	Unit and integration tests	<code>bun run test</code>
fast-check	Property-based testing	Used within Vitest
react-doctor	React best practices audit	<code>bunx react-doctor@lat</code>
Stryker	Mutation testing	<code>bun run stryker</code>
Playwright CT	Component testing in real browser	<code>bun run test:ct</code>

File Locations

- Unit/integration tests: `src/**/*.test.ts(x)`
- Property-based tests: alongside unit tests, using `fc.assert` / `fc.property`
- Playwright CT tests: `src/**/*.ct.tsx`

Coverage Thresholds

- Line coverage: 90%+
- Function coverage: 90%+
- Branch coverage: 90%+
- Statement coverage: 90%+
- Mutation score: 80%+

Test Patterns

- Tests must be deterministic: mock network calls, isolate state per test
- Use `@testing-library/react` for component tests
- Prefer small focused tests over large integration tests
- Use property-based tests for pure functions with well-defined input domain
- Arrange-Act-Assert structure
- One logical assertion per test

[code-style.md](#)

Code Style

TypeScript

- Strict mode enabled (strict: true in tsconfig)
- Never use any - use proper types, unknown, or generics
- Prefer explicit interfaces and type definitions
- No unused locals or parameters

Complexity Limits (enforced by Oxlint)

Rule	Limit
-----	-----
Cyclomatic complexity	10
Max lines per function	50
Max parameters	4
Max nesting depth	4
Max statements per function	15

Component Design

- One component per file
- Keep components under 80 lines
- Split components into small, focused units (single responsibility)
- Extract reusable logic into custom hooks
- Prefer composition over monolithic components

Styling

- Use TailwindCSS utility classes
- No inline styles
- Use clsx for conditional class composition

[commits.md](#)

Commit Workflow

Rules

- Use conventional commits: feat:, fix:, refactor:, test:, docs:, chore:
- One logical change per commit
- Commit only what was changed by the current step
- Always run lint + test before committing
- Step-by-step approach: finish one thing, verify, commit, move on

Before Every Commit

bun lint

```
bunx -y react-doctor@latest . --verbose --diff
bun run test
bun run stryker:incremental
bun run test:ct (if UI components changed)
```

Commit Message Format

type(scope): short description

Optional body explaining why, not what.

14. Config Files Reference

.gitignore additions

```
# Testing
coverage/
reports/
.stryker-tmp/

# Agent
.agent/sessions/

# Build
dist/
```

Full dependency list (devDependencies)

```
{
  "@playwright/experimental-ct-react": "^1.58.0",
  "@stryker-mutator/core": "^9.6.0",
  "@stryker-mutator/typescript-checker": "^9.6.0",
  "@stryker-mutator/vitest-runner": "^9.6.0",
  "@testing-library/dom": "^10.4.0",
  "@testing-library/jest-dom": "^6.9.0",
  "@testing-library/react": "^16.3.0",
  "@testing-library/user-event": "^14.6.0",
  "@vitest/coverage-v8": "^4.1.0",
  "@vitest/ui": "^4.1.0",
```

```
"fast-check": "^4.6.0",  
"jsdom": "^28.1.0",  
"lefthook": "^2.1.0",  
"oxfmt": "^0.41.0",  
"oxlint": "^1.56.0",  
"playwright": "^1.58.0",  
"vitest": "^4.1.0"  
}
```

Full scripts list

```
{  
  "dev": "vite dev",  
  "build": "vite build && tsc --noEmit",  
  "preview": "vite preview",  
  "test": "vitest run",  
  "test:ui": "vitest --ui",  
  "test:coverage": "vitest --coverage",  
  "lint": "oxlint --max-warnings=0",  
  "lint:fix": "oxlint --fix",  
  "format": "oxfmt",  
  "prepare": "lefthook install",  
  "stryker": "stryker run",  
  "stryker:incremental": "stryker run --incremental",  
  "test:ct": "playwright test -c playwright-ct.config.ts",  
  "test:ct:ui": "playwright test -c playwright-ct.config.ts --ui"  
}
```

Quick Start Checklist

For a new frontend project, follow these steps in order:

- ☐ Scaffold project with framework CLI
- ☐ Clean up demo files, switch to Bun
- ☐ Create `.agent/rules/` with 5 rule files
- ☐ Create `AGENTS.md`, symlink `CLAUDE.md`, `.claude`, `.codex`
- ☐ Install Oxlint + Oxfmt + Lefthook, create configs
- ☐ Install Vitest + fast-check + Testing Library, create configs
- ☐ Run `npx stryker init`, configure for vitest + typescript

- ☐ Install Playwright CT, create config, run `npx playwright install chromium`
 - ☐ Create `lefthook.yml` with all 6 hooks
 - ☐ Create `.github/workflows/publish.yml` with quality gates
 - ☐ Run `npx openspec init` for spec-driven development
 - ☐ Define first feature with `/openspec-propose`
 - ☐ Implement with TDD: `/openspec-apply`
 - ☐ Verify: coverage 90%+, mutation score 80%+, all CT tests pass
 - ☐ Commit. Ship. Repeat.
- 